

Session Knowledge Miner — Benutzeranleitung

Version: 1.0 | Stand: Mai 2026

Überblick

Der Session Knowledge Miner analysiert automatisch alle Claude Code-Sitzungen, extrahiert Probleme, Lösungen und Muster, und schreibt das Ergebnis als verknüpfte Markdown-Notizen in einen Obsidian-Vault. Ein Pre-Session-Hook injiziert relevante vergangene Lösungen als Kontext, bevor eine neue Konversation beginnt.

Komponenten

Komponente	Zweck
miner.py	CLI-Einstiegspunkt (mine, search, report, lessons)
src/parser.py	Liest JSONL-Transkripte aus ~/.claude/projects/
src/extractor.py	Erkennt Probleme, Lösungen, Befehle, Friction
src/embedder.py	Sentence-Transformer-Embeddings (Fallback: TF-IDF, Jaccard)
src/graph.py	SQLite-Wissensgraph (Sessions, Insights, Themen, Links)
src/obsidian.py	Schreibt Obsidian-Notizen mit [[wikilinks]]
src/hook_context.py	Pre-Session-Hook: injiziert vergangene Lösungen
config.json	Konfiguration (Pfade, Schwellenwerte)
bootstrap.ps1	Einmalige Ersteinrichtung

Installation (Ersteinrichtung)

Voraussetzungen

- Python 3.10 oder neuer
- Git (für Cross-PC-Sync)
- Obsidian (optional, aber empfohlen)

Schritt 1 — Bootstrap ausführen

```
cd C:\Users\reapertakashi\.claude\scripts\knowledge-miner
.\bootstrap.ps1
```

Das Skript:

1. Prüft Python-Installation
2. Installiert alle Abhängigkeiten (sentence-transformers, scikit-learn, etc.)
3. Erstellt das Datenverzeichnis
4. Erstellt den Obsidian-Vault-Ordner
5. Führt den ersten Mining-Durchlauf durch
6. Zeigt erste Lessons Learned

Schritt 2 — Obsidian öffnen

Den Ordner als Vault in Obsidian öffnen:

```
C:\Users\reapertakashi\OneDrive - Privatperson\Obsidian\Claude-Knowledge
```

In Obsidian: **Datei** → **Vault öffnen** → **Ordner als Vault öffnen**

Schritt 3 — Hooks sind bereits aktiv

Die Hooks wurden in `~/.claude/settings.json` eingetragen und laufen automatisch:

SessionEnd — `miner.py mine` nach jeder Session
UserPromptSubmit — `hook_context.py` vor jeder Eingabe

Konfiguration

Datei: `config.json`

```
{
  "transcripts_path": "C:\\Users\\...\\projects\\C--Users-...",
  "obsidian_vault_path": "C:\\Users\\...\\OneDrive - Privatperson\\Obsidian\\Claude-Knowledge",
  "data_dir": "C:\\Users\\...\\scripts\\knowledge-miner\\data",
  "similarity_threshold": 0.6,
  "theme_cluster_threshold": 0.7,
  "max_context_results": 3,
  "max_context_chars": 500,
  "language": "de+en"
}
```

Feld	Beschreibung	Standard
transcripts_path	Ordner mit Claude-JSONL-Transkripten	Pflichtfeld
obsidian_vault_path	Zielordner für Obsidian-Notizen	Pflichtfeld
data_dir	SQLite-DB und Embedding-Cache	./data
similarity_threshold	Mindestscore für Kontextinjizierung	0.6
theme_cluster_threshold	Schwelle für Theme-Clustering	0.7
max_context_results	Max. Treffer im Pre-Session-Hook	3
max_context_chars	Max. Zeichen im injizierten Kontext	500

CLI-Befehle

Alle Befehle werden im Projektordner ausgeführt:

```
cd C:\Users\reapertakashi\.claude\scripts\knowledge-miner
```

mine — *Transkripte verarbeiten*

```
python miner.py mine
```

Liest alle Sessions, aktualisiert den Wissensgraph, schreibt Obsidian-Notizen.

```
# Nur eine Session neu verarbeiten
python miner.py mine --session <session-id>
```

search — *Wissensgraph durchsuchen*

```
python miner.py search "PATH Fehler"
python miner.py search "tkinter StringVar"
```

Sucht semantisch über alle Insights. Gibt Typ, Titel und Inhalt zurück.

lessons — *Alle Erkenntnisse anzeigen*

```
python miner.py lessons
```

Listet alle Problem-Lösung-Paare aus allen Sessions.

themes — *Erkannte Muster anzeigen*

```
python miner.py themes
```

Zeigt alle automatisch erkannten Themen-Cluster mit Keywords und Session-Anzahl.

report — *Statistiken*

```
python miner.py report
```

Gibt Gesamtzahlen: Sessions, Insights, Themen, Links — aufgeschlüsselt nach Typ.

context — *Hook simulieren*

```
python miner.py context --prompt "tkinter Fehler beim Starten"
```

Simuliert was der Pre-Session-Hook ausgeben würde. Nützlich zum Testen.

Obsidian-Vault Struktur

```
Claude-Knowledge/
■■■ Sessions/
```

- ■■■ 2026-05-03 - PowerShell Script Manager.md
- ■■■ 2026-05-01 - Memory System Setup.md
- Themes/
- ■■■ GUI Entwicklung.md
- ■■■ MCP Konfiguration.md
- Lessons Learned.md

Session-Notiz (Aufbau)

Jede Session-Notiz enthält:

- Frontmatter** — session_id, Datum, Arbeitsverzeichnis, Tags
- Zusammenfassung** — Erster Absatz der Session
- Gelöste Probleme** — Problem + zugehörige Lösung, verlinkt mit Themes
- Verwendete Befehle** — Bis zu 8 wichtige Befehle
- Friction** — Hindernisse und Blocker
- Verknüpfte Sessions** — Ähnliche Sessions mit Ähnlichkeitsscore
- Muster / Themes** — Welchen Themen diese Session zugeordnet ist

Theme-Notiz (Aufbau)

- Keywords, die das Thema beschreiben
- Alle Sessions, die diesem Thema zugeordnet sind
- Beste bekannte Lösung aus allen Sessions

Lessons Learned (Aufbau)

Zentrales Index-Dokument, automatisch generiert:

- Geordnet nach Thema
- Alle Problem-Lösung-Paare aller Sessions
- Mit direkten `[[wikilinks]]` zu den jeweiligen Session-Notizen

Hooks — Automatischer Ablauf

SessionEnd-Hook (automatisches Mining)

Nach jeder Claude Code-Sitzung läuft automatisch:

```
python miner.py mine
```

Das neue Transkript wird geparkt, Insights extrahiert, Obsidian-Notizen aktualisiert.

Laufzeit: ca. 10–30 Sekunden je nach Session-Länge.

UserPromptSubmit-Hook (Kontext-Injizierung)

Vor jeder Eingabe sucht der Hook ähnliche vergangene Sessions:

```
[Knowledge Miner] Ähnliche vergangene Situationen / Similar past situations:
```

1. `**PowerShell Script Manager**` (84%)

Tkinter GUI mit StringVar-Validierung, executor.py Parameterprüfung

Diese Ausgabe erscheint als Systemkontext — Claude sieht die vergangenen Lösungen automatisch, ohne dass man explizit danach fragen muss.

Cross-PC-Sync

Was wird wie synchronisiert?

Inhalt	Methode	Pfad
Miner-Code + Config	Git (yannikits/claude-config)	~/.claude/scripts/knowledge-miner/
SQLite-DB + Embeddings	Nicht synchronisiert (lokal neu gebaut)	data/
Obsidian-Vault	OneDrive automatisch	OneDrive - Privatperson/Obsidian/Claude-Knowledge/
Claude-Settings + Hooks	Git (yannikits/claude-config)	~/.claude/settings.json

Einrichtung auf einem neuen PC

```
# 1. Config-Repo klonen
git clone https://github.com/yannikits/claude-config C:\Users\<name>\.claude

# 2. Miner-Abhängigkeiten installieren und ersten Mine-Durchlauf
cd C:\Users\<name>\.claude\scripts\knowledge-miner
.\bootstrap.ps1

# 3. Obsidian-Vault ist bereits via OneDrive vorhanden
#    Nur als Vault in Obsidian öffnen
```

Häufige Probleme

ModuleNotFoundError: No module named 'sentence_transformers'

```
python -m pip install sentence-transformers
```

Embedding-Backend zeigt tfidf statt sentence-transformers

Sentence-Transformers ist nicht installiert oder der Import schlägt fehl.

python miner.py report zeigt das aktuelle Backend.

Obsidian-Notizen werden nicht aktualisiert

1. Prüfen ob obsidian_vault_path in config.json korrekt ist
2. python miner.py mine manuell ausführen und Ausgabe prüfen

Hook läuft nicht

Prüfen ob die Einträge in ~/.claude/settings.json vorhanden sind:

SessionEnd — Eintrag mit miner.py mine

UserPromptSubmit — Eintrag mit hook_context.py

data/ **Ordner fehlt auf neuem PC**

```
python miner.py mine
```

Der Ordner und die DB werden automatisch erstellt.

Dateistruktur

```
knowledge-miner/
■■■■ config.json           # Konfiguration
■■■■ miner.py              # CLI-Einstiegspunkt
■■■■ bootstrap.ps1         # Ersteinrichtung
■■■■ requirements.txt       # Python-Abhängigkeiten
■■■■ .gitignore            # data/ ausgeschlossen
■■■■ docs/
■   ■■■■ Benutzeranleitung.md # Diese Datei
■   ■■■■ Benutzeranleitung.pdf
■■■■ src/
■   ■■■■ __init__.py
■   ■■■■ parser.py          # Transkript-Parser
■   ■■■■ extractor.py       # Insight-Extraktion
■   ■■■■ embedder.py        # Embedding-Backend
■   ■■■■ graph.py           # SQLite-Wissensgraph
■   ■■■■ obsidian.py        # Obsidian-Notizen-Generator
■   ■■■■ hook_context.py    # Pre-Session-Hook
■■■■ data/                 # Lokal, nicht in Git
    ■■■■ knowledge.db
    ■■■■ embeddings.pkl
```